
User Manual

for MPC560XB BASE Driver

Document Number: UM13BASEASR4.0R1.0.1
Rev 1.0





Contents

Section number	Title	Page
Chapter 1		
Revision History		
Chapter 2		
Introduction		
2.1	Supported Derivatives.....	7
2.2	Overview.....	8
2.3	About this Manual.....	8
2.4	Acronyms and Definitions.....	8
2.5	Reference List.....	9
2.6	Symbolic Names DISCLAIMER.....	9
Chapter 3		
Macros used in MCAL.h		
Chapter 4		
Define and Macros Definitions used in Reg_Macros.h		
4.1	Bit Definitions.....	13
4.2	Define REG_WRITE8.....	14
4.3	Define REG_WRITE16.....	14
4.4	Define REG_WRITE32.....	14
4.5	Define REG_READ8.....	14
4.6	Define REG_READ16.....	15
4.7	Define REG_READ32.....	15
4.8	Define REG_AWRITE8.....	15
4.9	Define REG_AWRITE16.....	15
4.10	Define REG_AWRITE32.....	15
4.11	Define REG_AREAD8.....	16
4.12	Define REG_AREAD16.....	16
4.13	Define REG_AREAD32.....	16
4.14	Define REG_BIT_CLEAR8.....	16

Section number	Title	Page
4.15	Define REG_BIT_CLEAR16.....	16
4.16	Define REG_BIT_CLEAR32.....	16
4.17	Define REG_BIT_GET8.....	17
4.18	Define REG_BIT_GET16.....	17
4.19	Define REG_BIT_GET32.....	17
4.20	Define REG_BIT_SET8.....	17
4.21	Define REG_BIT_SET16.....	17
4.22	Define REG_BIT_SET32.....	18
4.23	Define REG_RMW8.....	18
4.24	Define REG_RMW16.....	18
4.25	Define REG_RMW32.....	18

Chapter 1

Revision History

Table 1-1. Revision History

Revision	Date	Author	Description
1.0	14.04.2014	Hoang Ngoc Hien	First version for RTM1.0.1 release



Chapter 2

Introduction

This User Manual lists the macros which are defined in BASE for MPC560XB module and have chances of being redefined in other drivers.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of Freescale Semiconductor:

Table 2-1. MPC560XB Supported Derivatives

Freescale Semiconductor	mpc5601dxlh_lqfp64 mpc5601dxll_lqfp100 mpc5602dxlh_lqfp64 mpc5602dxll_lqfp100 mpc5602bxlh_lqfp64 mpc5602bxll_lqfp100 mpc5602bxlq_lqfp144 mpc5602cxlh_lqfp64 mpc5602cxll_lqfp100 mpc5603bxlh_lqfp64 mpc5603bxll_lqfp100 mpc5603bxlq_lqfp144 mpc5603cxlh_lqfp64 mpc5603cxll_lqfp100 mpc5604bxlh_lqfp64 mpc5604bxll_lqfp100 mpc5604bxlq_lqfp144 mpc5604bxmg_mapbga208 mpc5604cxlh_lqfp64 mpc5604cxll_lqfp100 mpc5605bxll_lqfp100 mpc5605bxlq_lqfp144 mpc5605bxlu_lqfp176 mpc5606bkll_lqfp100 mpc5606bxlq_lqfp144 mpc5606bxlu_lqfp176 mpc5607bxlu_lqfp176 mpc5607bxmg_mapbga208
-------------------------	--

All of the above microcontroller devices are collectively named as **MPC560XB**.

2.2 Overview

AUTOSAR (AUTOmotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About this Manual

This Technical Reference employs the following typographical conventions:

Boldface type: Bold is used for important terms, notes and warnings.

Italic font: Italic typeface is used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

2.4 Acronyms and Definitions

Table 2-2. Acronyms and Definitions

Term	Definition
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
DEM	Diagnostic Event Manager
DET	Development Error Tracer
ECU	Electronic Control Unit
GPIO	General Purpose Input Output
GPT	General Purpose Timer
MCAL	Microntroller Abstraction Layer
MIDE	Multi Integrated Development Environment
PWM	Pulse Width Modulation
VLE	Variable Length Encoding
XML	Extensible Markup Language

2.5 Reference List

Table 2-3. Reference List

Title	Version
MPC5607B Reference Manual	Rev. 7.2, May 2012
MPC5604B Reference Manual	Rev. 8.2, September 2013
5606B Reference Manual	Rev. 2, May 2014
5602D Reference Manual	Rev. 4.2, Dec 2013

2.6 Symbolic Names DISCLAIMER

All containers having the symbolic name tag set as true in the Autosar schema will generate defines like:

```
#define <Container_Short_Name> <Container_ID>
```

For this reason it is forbidden to duplicate the name of such containers across the MCAL configuration, or to use names that may trigger other compile issues (e.g. match existing `#ifdefs` arguments).

Chapter 3

Macros used in MCAL.h

This table contains the macros defined in Mcal.h. It gives compiler wise definitions of the assembly language instructions. The covered compilers are GHS, Windriver DIAB, CodeWarrior and High Tec

Table 3-1. Macro List

Compiler	Macro	Value
GHS	ASM_KEYWORD	__asm
	EXECUTE_WAIT()	ASM_KEYWORD(" wait")
	ALIGNED_VARS_START(sec_name, align)	
	VAR_ALIGN(v, size)	__attribute__((aligned(size))) v;
	ALIGNED_VARS_STOP()	'0'
WindRiver Diab	ASM_KEYWORD	asm volatile
	EXECUTE_WAIT()	ASM_KEYWORD(" .word 0x7C00007C")
	ALIGNED_VARS_START(sec_name, align)	
	VAR_ALIGN(v, size)	__attribute__((aligned(size))) v;
	ALIGNED_VARS_STOP()	
CodeWarrior	ASM_KEYWORD	asm
	EXECUTE_WAIT()	ASM_KEYWORD (" opword 0x7C00007C")
	ALIGNED_VARS_START(sec_name, align)	
	VAR_ALIGN(v, size)	v __attribute__((aligned(size)));
	ALIGNED_VARS_STOP()	
High Tec	ASM_KEYWORD	__asm
	EXECUTE_WAIT()	ASM_KEYWORD(" wait")
	ALIGNED_VARS_START(sec_name, align)	
	VAR_ALIGN(v, size)	__attribute__((aligned(size))) v;
	ALIGNED_VARS_STOP()	



Chapter 4

Define and Macros Definitions used in Reg_Macros.h

Data defines supported by the Reg_Macros.h file in BASE modules are listed below.

4.1 Bit Definitions

The following table shows Macros used for bit(0-32) definitions.

Table 4-1. BIT Definitions

Compiler	Macro
BIT0	0x00000001UL
BIT1	0x00000002UL
BIT2	0x00000004UL
BIT3	0x00000008UL
BIT4	0x00000010UL
BIT5	0x00000020UL
BIT6	0x00000040UL
BIT7	0x00000080UL
BIT8	0x00000100UL
BIT9	0x00000200UL
BIT10	0x00000400UL
BIT11	0x00000800UL
BIT12	0x00001000UL
BIT13	0x00002000UL
BIT14	0x00004000UL
BIT15	0x00008000UL
BIT16	0x00010000UL
BIT17	0x00020000UL
BIT18	0x00040000UL
BIT19	0x00080000UL

Table continues on the next page...

Table 4-1. BIT Definitions (continued)

Compiler	Macro
BIT20	0x00100000UL
BIT21	0x00200000UL
BIT22	0x00400000UL
BIT23	0x00800000UL
BIT24	0x01000000UL
BIT25	0x02000000UL
BIT26	0x04000000UL
BIT27	0x08000000UL
BIT28	0x10000000UL
BIT29	0x20000000UL
BIT30	0x40000000UL
BIT31	0x80000000UL

4.2 Define REG_WRITE8

A macro for 8 bits memory write .

Definition: `#define REG_WRITE8(address, value) ((* (volatile uint8*) (address)) = (uint8) (value))`

4.3 Define REG_WRITE16

A macro for 16 bits memory write .

Definition: `#define REG_WRITE16(address, value) ((* (volatile uint16*) (address)) = (uint16) (value))`

4.4 Define REG_WRITE32

A macro for 32 bits memory write .

Definition: `#define REG_WRITE32(address, value) ((* (volatile uint32*) (address)) = (uint32) (value))`

4.5 Define REG_READ8

A macro for 8 bits memory read .

Definition: `#define REG_READ8(address) (*(volatile uint8*)(address))`

4.6 Define REG_READ16

A macro for 16 bits memory read .

Definition: `#define REG_READ16(address) (*(volatile uint16*)(address))`

4.7 Define REG_READ32

A macro for 32 bits memory read .

Definition: `#define REG_READ32(address) (*(volatile uint32*)(address))`

4.8 Define REG_AWRITE8

8 bits indexed memory write macro.

Definition: `#define REG_AWRITE8(address, i, value) (REG_WRITE8 ((address)+(uint32)(i), (value)))`

4.9 Define REG_AWRITE16

16 bits indexed memory write macro.

Definition: `#define REG_AWRITE16(address, i, value) (REG_WRITE16((address)+((uint32)((uint32)(i))<<1U)), (value)))`

4.10 Define REG_AWRITE32

32 bits indexed memory write macro.

Definition: `#define REG_AWRITE32(address, i, value) (REG_WRITE32((address)+((uint32)((uint32)(i))<<2U)), (value)))`

4.11 Define REG_AREAD8

8 bits indexed memory read macro.

Definition: `#define REG_AREAD8(address, i) (REG_READ8 ((address)+(uint32)(i)))`

4.12 Define REG_AREAD16

16 bits indexed memory read macro

Definition: `#define REG_AREAD16(address, i) (REG_READ16 ((address)+((uint32)((uint32)(i))<<1U)))`

4.13 Define REG_AREAD32

32 bits indexed memory read macro.

Definition: `#define REG_AREAD32(address, i) (REG_READ32 ((address)+((uint32)((uint32)(i))<<2U)))`

4.14 Define REG_BIT_CLEAR8

8 bit bits clearing macro.

Definition: `#define REG_BIT_CLEAR8(address, mask) ((*volatile uint8*)(address)) &= ((uint8)~((uint8)(mask)))`

4.15 Define REG_BIT_CLEAR16

16 bit bits clearing macro.

Definition: `#define REG_BIT_CLEAR16(address, mask) ((*volatile uint16*)(address)) &= ((uint16)~((uint16)(mask)))`

4.16 Define REG_BIT_CLEAR32

32 bit bits clearing macro.

Definition: `#define REG_BIT_CLEAR32(address, mask) ((* (volatile uint32*) (address)) &= ((uint32) ~ ((uint32) (mask))))`

4.17 Define REG_BIT_GET8

8 bit bits getting macro.

Definition: `#define REG_BIT_GET8(address, mask) ((* (volatile uint8*) (address)) & (uint8) (mask))`

4.18 Define REG_BIT_GET16

16 bit bits getting macro.

Definition: `#define REG_BIT_GET16(address, mask) ((* (volatile uint16*) (address)) & (uint16) (mask))`

4.19 Define REG_BIT_GET32

32 bit bits getting macro.

Definition: `#define REG_BIT_GET32(address, mask) ((* (volatile uint32*) (address)) & (uint32) (mask))`

4.20 Define REG_BIT_SET8

8 bit bits setting macro.

Definition: `#define REG_BIT_SET8(address, mask) ((* (volatile uint8*) (address)) |= (uint8) (mask))`

4.21 Define REG_BIT_SET16

16 bit bits setting macro.

Definition: `#define REG_BIT_SET16(address, mask) ((* (volatile uint16*) (address)) |= (uint16) (mask))`

4.22 Define REG_BIT_SET32

32 bit bits setting macro.

Definition: `#define REG_BIT_SET32(address, mask) ((* (volatile uint32*) (address)) |= (uint32) (mask))`

4.23 Define REG_RMW8

8 bit clear bits and set with new value.

Definition: `#define REG_RMW8(address, mask, value) (REG_WRITE8((address), ((REG_READ8(address) & ((uint8)~((uint8)(mask)))) | ((uint8)(value)))))`

4.24 Define REG_RMW16

16 bit clear bits and set with new value.

Definition: `#define REG_RMW16(address, mask, value) (REG_WRITE16((address), ((REG_READ16(address) & ((uint16)~((uint16)(mask)))) | ((uint16)(value)))))`

4.25 Define REG_RMW32

32 bit clear bits and set with new value.

Definition: `#define REG_RMW32(address, mask, value) (REG_WRITE32((address), ((REG_READ32(address) & ((uint32)~((uint32)(mask)))) | ((uint32)(value)))))`

How to Reach Us:**Home Page:**freescale.com**Web Support:**freescale.com/support

Information in this document is provided solely to enable system and software implementers to use Freescale products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document.

Freescale reserves the right to make changes without further notice to any products herein. Freescale makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does Freescale assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in Freescale data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. Freescale does not convey any license under its patent rights nor the rights of others. Freescale sells products pursuant to standard terms and conditions of sale, which can be found at the following address: freescale.com/SalesTermsandConditions.

Freescale, the Freescale logo, Altivec, C-5, CodeTest, CodeWarrior, ColdFire, ColdFire+, C-Ware, Energy Efficient Solutions logo, Kinetis, mobileGT, PowerQUICC, Processor Expert, QorIQ, Qorivva, StarCore, Symphony, and VortiQa are trademarks of Freescale Semiconductor, Inc., Reg. U.S. Pat. & Tm. Off. Airfast, BeeKit, BeeStack, CoreNet, Flexis, Layerscape, MagniV, MXC, Platform in a Package, QorIQ Qonverge, QUICC Engine, Ready Play, SafeAssure, SafeAssure logo, SMARTMOS, Tower, TurboLink, Vybrid, and Xtrinsic are trademarks of Freescale Semiconductor, Inc. All other product or service names are the property of their respective owners.

© 2014 Freescale Semiconductor, Inc.

Document Number UM13BASEASR4.0R1.0.1
Revision 1.0